模块七：FastAPI Web 服务

学习目标

- 理解为什么 AI 应用需要 Web Service 层
- 掌握 FastAPI 核心概念（路由、请求模型、依赖注入、中间件）
- 学会用 Pydantic 定义 API 数据模型
- 实现 SSE 流式响应
- 了解 API 设计最佳实践

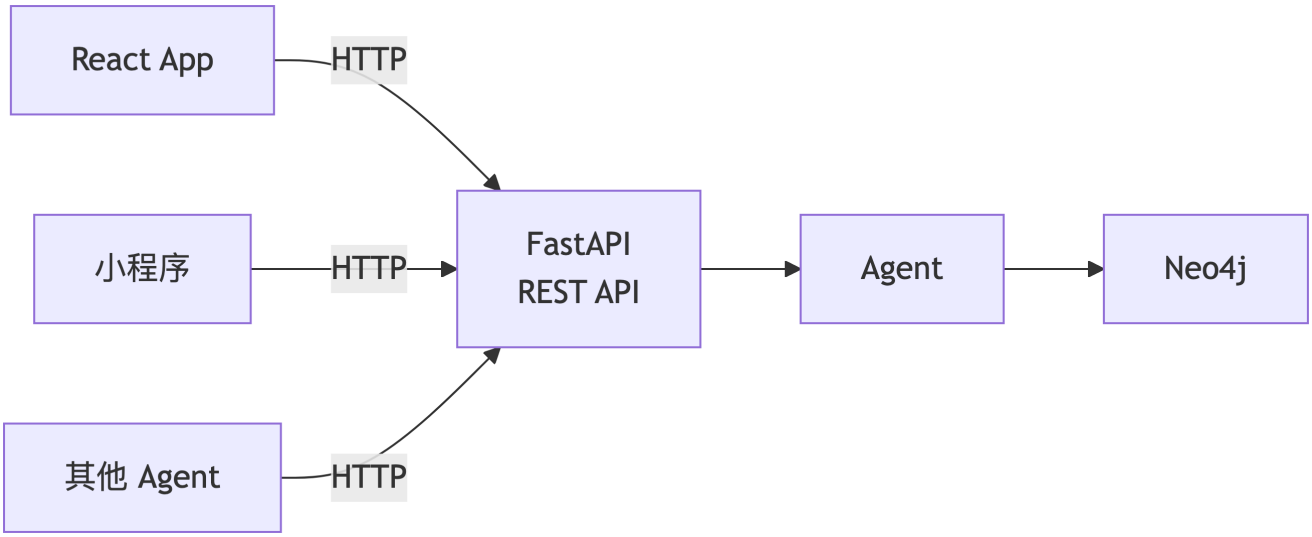
7.1 为什么需要 Web 服务？

CLI vs Streamlit vs REST API 对比

特性	CLI (`app.py`)	Streamlit (`streamlit_app.py`)	REST API (`api_server.py`)
用户界面	终端命令行	浏览器 Web UI	无（程序调用）
适合场景	开发调试	教学演示	生产部署、前后端分离
并发支持	单用户	单用户/少量用户	高并发
可集成性	低	低	高（任何语言可调用）
前端自由度	无	Streamlit 组件限制	完全自由
自动化测试	困难	困难	简单（HTTP 请求）

核心观点

> **REST API 是 AI 应用的标准交付方式。**
>
> 无论前端用 React、Vue、小程序还是其他 Agent 调用，都通过 HTTP API 与后端通信。



7.2 FastAPI 核心概念

7.2.1 路由（Routing）

路由将 URL 路径映射到处理函数:

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/api/v1/health")           # GET 请求
async def health():
    return {"status": "healthy"}

@app.post("/api/v1/chat")            # POST 请求
async def chat(request: ChatRequest):
    return {"response": "..."}
```

HTTP 方法与语义:

方法	语义	示例
GET	获取资源	`GET /api/v1/graph/rooms`
POST	创建/执行	`POST /api/v1/chat`
PUT	完整更新	`PUT /api/v1/settings`
DELETE	删除资源	`DELETE /api/v1/sessions/{id}`

7.2.2 请求/响应模型（Pydantic）

FastAPI 使用 Pydantic 模型进行自动验证和文档生成:

```
from pydantic import BaseModel, Field

class ChatRequest(BaseModel):
    """请求模型 — 定义客户端必须发送什么"""
    message: str = Field(
        ...,                        # ... 表示必填
        min_length=1,               # 最短 1 字符
        max_length=1000,            # 最长 1000 字符
        description="用户的自然语言请求",
    )
    debug: bool = Field(default=False)
    session_id: Optional[str] = None

class ChatResponse(BaseModel):
    """响应模型 — 定义服务端返回什么"""
    response: str
    reasoning_trace: list[str] = []
    metadata: dict = {}
```

Pydantic 的三个作用: 1. 类型验证 — 自动拒绝格式错误的请求（返回 422） 2. 文档生成 — 自动生成 JSON Schema → Swagger UI 展示 3. 序列化 — 自动将 Python 对象转为 JSON 响应

7.2.3 依赖注入（Dependency Injection）

FastAPI 的 `Depends()` 模式将资源管理与业务逻辑分离:

```
from fastapi import Depends

def get_agent():
    """依赖函数：返回共享的 Agent 实例"""
    if app_state.agent is None:
        raise HTTPException(status_code=503, detail="Agent not ready")
    return app_state.agent

@app.post("/api/v1/chat")
async def chat(
    request: ChatRequest,
    agent=Depends(get_agent),      # ← 自动注入 Agent
):
    response = agent.run(request.message)
    return {"response": response}
```

为什么用依赖注入？- 端点函数不需要知道 Agent 如何创建 - 测试时可以替换为 Mock Agent - 资源初始化失败时统一返回 503

7.2.4 生命周期管理（Lifespan）

用 `lifespan` 上下文管理器处理启动和关闭逻辑:

```
from contextlib import asynccontextmanager

@asynccontextmanager
async def lifespan(app: FastAPI):
    # === 启动时执行 ===
    app_state.agent = SmartHomeAgent()
    app_state.connection = get_connection()

    yield  # 应用运行中

    # === 关闭时执行 ===
    close_connection()

app = FastAPI(lifespan=lifespan)
```

7.2.5 中间件（Middleware）

中间件包裹每个请求/响应的处理过程:

我们实现了两个教学级中间件:

```
# 请求日志中间件
class RequestLoggingMiddleware(BaseHTTPMiddleware):
    async def dispatch(self, request, call_next):
        start = time.time()
        response = await call_next(request)
        duration = time.time() - start
        logger.info(f"{request.method} {request.url.path}
{response.status_code} ({duration:.3f}s)")
        return response

# 令牌桶限流中间件
class RateLimitMiddleware(BaseHTTPMiddleware):
    async def dispatch(self, request, call_next):
        if over_limit(client_ip):
            return JSONResponse(status_code=429, content={"error":
"rate_limit_exceeded"})
        return await call_next(request)
```

7.3 API 设计实践

RESTful 原则

我们的 API 遵循 RESTful 设计:

```
POST /api/v1/chat                      # 执行聊天
POST /api/v1/chat/stream               # 流式聊天
GET  /api/v1/health                    # 健康检查
GET  /api/v1/graph/stats               # 图谱统计
GET  /api/v1/graph/rooms               # 房间列表
GET  /api/v1/graph/rooms/{name}/devices # 房间设备
GET  /api/v1/graph/devices             # 设备列表
GET  /api/v1/graph/scenes              # 场景列表
```

设计要点: - 版本控制: `/api/v1/` 前缀，方便未来升级到 v2 - 资源命名: 使用名词（`rooms`、`devices`），不用动词 - 嵌套资源: `/rooms/{name}/devices` 表示房间包含设备

错误处理

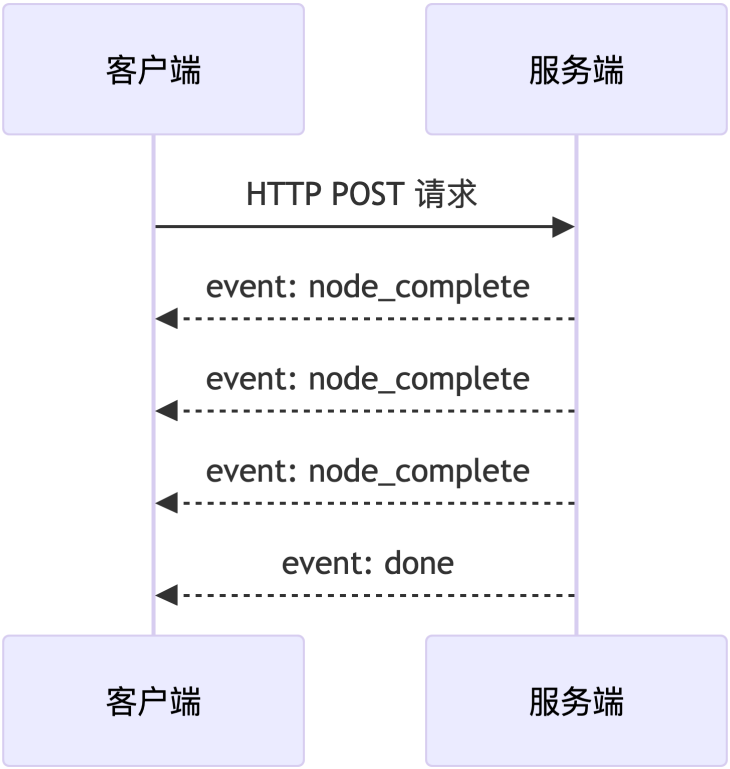
```python
# FastAPI 自动处理验证错误 (422)
# 手动抛出业务错误：
raise HTTPException(
    status_code=404,
    detail="Room 'kitchen' not found",
)

# 标准错误响应格式
{
    "error": "not_found",
    "detail": "Room 'kitchen' not found"
}
```

7.4 流式响应（SSE）

什么是 Server-Sent Events?

SSE 是一种服务器向客户端推送事件的协议:



SSE vs WebSocket:

特性	SSE	WebSocket
方向	服务端 → 客户端（单向）	双向
协议	HTTP	独立协议
复杂度	简单	较复杂
适用场景	AI 流式输出、通知	聊天、游戏

实现：节点级流式

我们的 Agent 工作流有多个节点，每个节点完成时发送一个事件:

```
from sse_starlette.sse import EventSourceResponse

@app.post("/api/v1/chat/stream")
async def chat_stream(request: ChatRequest, agent=Depends(get_agent)):
    async def event_generator():
        for event in agent.run_streaming(request.message):
            for node_name, node_data in event.items():
                yield {
```

```
                    "event": "node_complete",
                    "data": json.dumps({
                        "node_name": node_name,
                        "data": node_data,
                        "timestamp": datetime.now().isoformat(),
                    }),
                }
        yield {"event": "done", "data": json.dumps({"status": "complete"})}

    return EventSourceResponse(event_generator())
```

前端消费 SSE（JavaScript 示例）

```
const eventSource = new EventSource('/api/v1/chat/stream', {
    method: 'POST',
    body: JSON.stringify({ message: '打开客厅灯' }),
});

eventSource.addEventListener('node_complete', (e) => {
    const data = JSON.parse(e.data);
    console.log(`节点 ${data.node_name} 完成:`, data.data);
});

eventSource.addEventListener('done', () => {
    console.log('处理完成');
    eventSource.close();
});
```

Python 消费 SSE

```
import httpx

with httpx.stream("POST", "http://localhost:8000/api/v1/chat/stream",
                  json={"message": "打开客厅灯"}) as response:
    for line in response.iter_lines():
        if line.startswith("data:"):
            data = json.loads(line[5:])
            print(f"[{data['node_name']}] {data.get('data', {})}")
```

7.5 生产级中间件

日志中间件

```
INFO: POST /api/v1/chat 200 (0.523s)
INFO: GET /api/v1/health 200 (0.012s)
INFO: POST /api/v1/chat 429 (0.001s)   # 被限流
```

日志中间件还在响应头中添加了处理时间:

```
X-Process-Time: 0.523
```

限流中间件

使用令牌桶算法（Token Bucket）:

```
每个客户端 IP → 60秒窗口内最多 30 个请求
超过限制 → 返回 HTTP 429 Too Many Requests
```

注意: 这是教学级实现（内存存储）。生产环境应使用 Redis 实现分布式限流。

CORS 中间件

```
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],      # 允许所有域名（开发用）
    allow_methods=["*"],      # 允许所有 HTTP 方法
    allow_headers=["*"],      # 允许所有请求头
)
```

7.6 API 文档与测试

Swagger UI（自动生成）

启动服务器后，访问 `http://localhost:8000/docs` 查看交互式 API 文档。

FastAPI 自动从 Pydantic 模型和函数文档生成 Swagger UI，支持直接在浏览器中测试 API。

命令行测试

```
# 健康检查
curl http://localhost:8000/api/v1/health | python -m json.tool

# 聊天（同步）
curl -X POST http://localhost:8000/api/v1/chat \
```

```
  -H "Content-Type: application/json" \
  -d '{"message": "打开客厅灯", "debug": true}'

# 查看图谱统计
curl http://localhost:8000/api/v1/graph/stats

# 列出所有房间
curl http://localhost:8000/api/v1/graph/rooms

# 查看某房间的设备
curl http://localhost:8000/api/v1/graph/rooms/living/devices
```

Python 测试

```
import httpx

client = httpx.Client(base_url="http://localhost:8000")

# 健康检查
r = client.get("/api/v1/health")
print(r.json())

# 聊天
r = client.post("/api/v1/chat", json={
    "message": "把客厅灯调暗一点",
    "debug": True,
})
data = r.json()
print(f"回复: {data['response']}")
print(f"推理过程: {data['reasoning_trace']}")
```

7.7 动手实验

请打开 `notebooks/05_fastapi_web_service.ipynb`，完成以下实验:

1. **启动 API Server**: 在终端运行 `python api_server.py`
2. **浏览 Swagger UI**: 访问 http://localhost:8000/docs
3. **用 httpx 调用各端点**: 健康检查、聊天、图谱查询
4. **测试 SSE 流式响应**: 观察节点级事件
5. **测试限流**: 快速发送多个请求，观察 429 响应

关键代码文件

文件	内容
`src/api/models.py`	Pydantic 请求/响应模型
`src/api/server.py`	FastAPI 应用和端点
`src/api/middleware.py`	日志、限流中间件
`api_server.py`	API 启动入口

下一节：08_Docker 容器化部署.qmd